NMAM Institute of Technology
Department of Information Science and Engineering
Semester 7

UI Design Course Project Report

Presence

A Role-Based Attendance Management Web Application

Course: IS3101, User Interface Design
Course Faculty: Dr. Jason Elroy Martis

Submitted by
Manvith D Shettigar
USN: NNM23IS094

Date: 30-10-2026

1. Problem Statement and Objectives

1.1 Understanding the Problem

Attending in a representative college classroom still happens on paper registers or slow portals, in actual application; mistakes are happening easily during the manual counting. Going through a list of 60 students by roll number takes up five to ten minutes of every single class, in real-world scenarios; teachers have to sit and count the total classes held, add up each student's attended hours, and find percentages by hand to prepare the shortage list, when the dust settles of the semester. As, well, students often have no idea their attendance is below 75% until the final catch goes up right before exams.

Switching to fingerprint machines or QR codes sounds good on paper, but they almost always run into the issues in a real classroom, in many respects; a long line at the door is being caused by biometric scanners, and QR codes either fail when the room's Wi-Fi drops or get shared instantly in hostel group chats, on the whole.

To resolve this, I made **Presence**, an attendance web app that's fitting normal day-to-day lectures, and is skipping phone scanning and unnecessary labels like "Late." A student is simply marked Present or Absent, with an On-Duty (OD) option available when someone misses class for placements, sports, or college fests, to a significant degree. I devised the app employing **React 18**, **TypeScript**, and **Tailwind CSS**, the web styling language, broadly speaking.

1.2 Targets of the Project

The aims of the project are:

- Designing a role-based portal with three separate views, including Student, Faculty (Lecturer), and Master Admin, on the whole.
- Building a fast marking screen where a lecturer can, to some extent, likely finish attendance for 60 students in, under a minute, to a significant degree.
- Setting all students to "Present" by default so the teacher only has to tap the roll numbers of the absentees, to a significant degree.
- Supporting both 1-hour theory classes and 2-hour lab sessions so double slots are, after all, counted without attending twice, broadly speaking.
- Creating an in-app attendance register table so teachers and admins can view all dates and percentages on screen without downloading a file first, in actual application.
- Offering a one-click CSV export formatted for the college office, with an option to filter only shortage students (< 75%), broadly speaking.
- Showing students their live percentage along with their "Bunk Buffer" (classes they can safely miss) or "Recovery Target" (classes they must attend in a row to reach 75%).

- Starting the app completely blank without fake dummy data, employing a first-time setup wizard so any department can enter their own batches from scratch, in real-world scenarios.

1.3 Scope

Presence is an academic project built to show a fast, concrete attendance workflow for engineering departments at NMAM Institute of Technology [1], in many respects; it isn't an officially linked ERP of NMAMIT. But instead of hardcoding one precise batch, the application starts clean and lets the Master Admin create departments, enroll student USNs via bulk CSV paste, and allot subjects for any semester from I to VIII, in many respects; all entries are stored locally in the browser employing localStorage keys [7], so the app runs as a self-contained client without needing an external database server or email service during the check.

2. System Overview and Architecture

2.1 Background and Rationale

A single-page application (SPA) architecture was chosen employing React 18 [2] and TypeScript [4], bundled with Vite [3], to a significant degree. As, classroom tools are needing to respond immediately when a teacher taps 60 roll numbers in a row, keeping state transitions on the client removes network lag between clicks, in real-world scenarios. Data is persisted across sessions in the browser's localStorage [7] through typed helper functions, and the entire application is structured so that the storage layer can be swapped with a REST API or database later without changing the UI components.

2.2 Technology Stack

Table 1: Technology stack of Presence

Layer	Technology	Purpose
Front end	React 18, TypeScript, Vite	Component state, strict data typing, fast local build and HMR [2][3][4]
Styling	Tailwind CSS v3, PostCSS	Responsive layout, sticky table columns, class-based dark mode [5]
Icons	lucide-react	Clean icons for navigation, status badges, and action buttons [6]
Typography	Fraunces (serif), Inter (sans), Monospace	Clear hierarchy for headings, body text, and USN roll numbers
Storage	Browser localStorage API	Persistent JSON storage for users, courses, sessions and attendance [7]
Exports	Browser Blob and URL.createObjectURL	Client-side CSV register generation for Shine and Google Sheets [8]

2.3 Architecture

Every action in the app follows a predictable state flow; in turn, if isInitialized is false, the app locks into the SetupWizard, broadly speaking; once set up, the user's part and identifier (Admin Email, Faculty ID, or Student USN) are being checked by the LoginPortal and sets the active session; this flow is shown by figure 1, on the whole. When the app loads, the saved JSON arrays were read from localStorage by custom storage hooks.

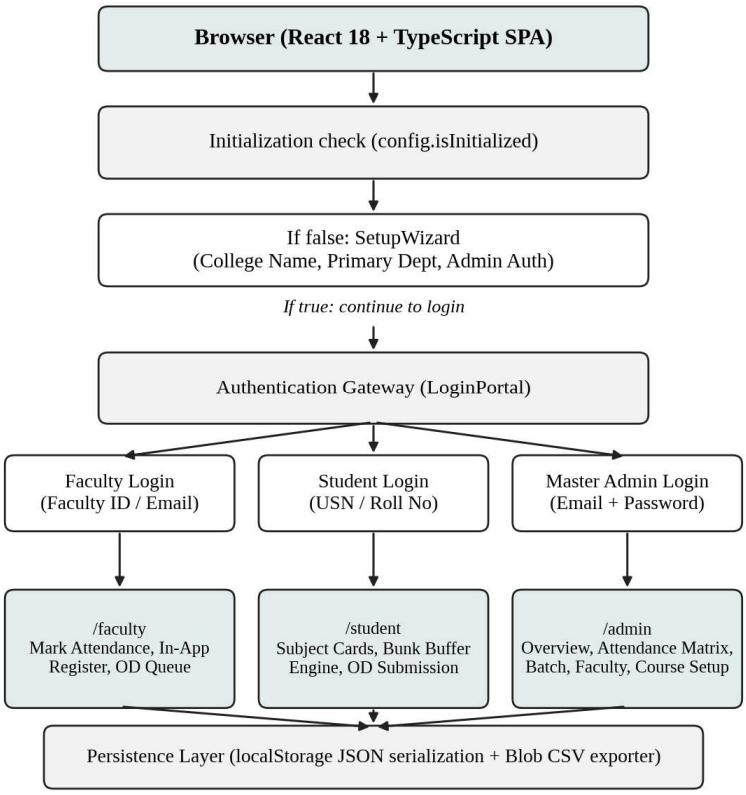


Figure 1, including *Application state flow and role-scoped architecture of Presence*

2.4 Project Structure

Table 2: Principal files and modules

File / Module	Contents
src/App.tsx	Central application state, TypeScript interfaces, routing, and all role modules
SetupWizard	First-launch configuration for college name, primary department, and threshold
LoginPortal	Role-switched login card for Faculty, Student and Master Admin
FacultyMarkingView	Roll number grid, list view, 1-hr/2-hr slot weight, and sticky Save Changes bar
AttendanceRegisterView	In-app horizontal spreadsheet table with sticky USN columns and CSV export

File / Module	Contents
StudentDashboard	Subject attendance cards, progress bars, Bunk Buffer, and Recovery Target logic
Admin□Managers	Bulk CSV student enrollment, faculty onboarding, and course allotment screens
tailwind.config.js	Tailwind v3 configuration with darkMode: 'class' allowed

3. User Roles and Access Control

3.1 The Three Portals

Table 3, such as Functions and their capabilities

Role	Login Identifier	Main Capabilities
Student	University USN (e.g., NNM23IS001) or college email	View subject-wise attendance %, check Bunk Buffer / shortage alerts, turn in On-Duty (OD) or medical leave requests
Faculty	Faculty ID (e.g., FAC-101) or email	Do 1-hr or 2-hr sessions for allotted courses, mark attendance via Roll Number Grid, view in-app register, approve/reject OD requests, export CSV
Master Admin	Master Admin email and password set during the initialization	Manage departments, onboard faculty, bulk-import student USNs, allot courses to faculty, set minimum attendance threshold, inspect all registers

3.2 Permission Matrix

Table 4: Permission matrix

Capability	Student	Faculty	Master Admin
View own attendance and Bunk Buffer	Read	—	—
Submit OD / Medical leave request	Write	—	—
Mark or save class attendance	No	Allotted courses only	No
Approve or reject OD requests	No	Allotted courses only	No
View In-App Attendance Register	No	Allotted courses only	All courses in college
Download CSV Register	No	Allotted courses only	All courses in college
Enroll students (single or bulk CSV)	No	No	Yes
Onboard faculty and allot courses	No	No	Yes

Capability	Student	Faculty	Master Admin
Change attendance threshold (e.g., 75%)	No	No	Yes

3.3 Scope Enforcement in Code

Their part and userId are saved in currentUser; thus, the navigation bar and primary workspace render only the components permitted for that part, when a user logs in. As well, course lists are filtered so a lecturer only sees subjects allotted to their facultyId:

```
{currentUser.role === 'FACULTY' && activeTab === 'dashboard' && (
  <FacultyMarkingView
    faculty={faculty.find((f) => f.id === currentUser.userId)!}
    courses={courses.filter((c) => c.facultyId === currentUser.userId)}
    students={students}
    onSaveSession={(newSession, newRecords) => {
      setSessions((prev) => [...prev, newSession]);
      setAttendance((prev) => [...prev, ...newRecords]);
    }}
  />
)}
```

4. Data Model and Storage Design

4.1 Collections and Interfaces

The data model is split into the eight clean TypeScript interfaces, grouped into the seven collections and stored, under the separate localStorage keys [7], in real-world scenarios.

Table 5 - Data collections and interfaces

Group	Interface / Key	Remarks
Configuration	SystemConfig (presence_inst_config)	Stores institution name, attendance threshold (default 75%), and isInitialized flag
Departments	Department (presence_inst_depts)	Branch name and code (such as ISE, CSE, ECE)
People	Student, Faculty	USN, semester and branch for students; facultyId and branch for lecturers
Curriculum	Course (presence_inst_courses)	Subject code (e.g. 21CS51), title, semester (1-8), department, and designated facultyId
Sessions	Session (presence_inst_sessions)	Conducted class date, hour weight (1 for theory, 2 for lab), and optional lecture topic
Attendance	AttendanceRecord (presence_inst_attendance)	Links sessionId and studentId with status ‘P’, ‘A’, or ‘OD’
Leave / OD	LeaveRequest (presence_inst_leaves)	Student OD or medical request with date and reason, with a concentrate on status (PENDING, Authorized, REJECTED)

4.2 Why a Binary Model Was Chosen

I thought about putting in a 30-second rotating QR code and four separate buttons for Present, Absent, Late and Excused, when I first started planning the app, to a significant degree. What's more, university rules don't count “Late” as half-attendance anyway, you either get the attendance for that hour or you don't. Once I thought about how our own classes run, I realized no lecturer wants 60 students phoning during a lecture, in actual application; indeed, scanning and the “Late” button made the interface much less cluttered and much faster for a teacher to use, before starting a lecture.

5. Institutional Setup and Authentication

5.1 First-Launch Setup Wizard

Because, all hardcoded dummy data was removed, opening the application for the first time launches the **Institutional Setup Wizard**, to a significant degree; the administrator enters the college name (e.g., *N.M.A.M. Institute of Technology*), the primary department name and code (*Information Science*, ISE), the Master Admin email and password, and the needed attendance percentage (75%), in many respects. Turning in this form sets up the storage and opens the login gateway.

5.2 Role-Switched Login Portal

The login page has a three-way tab switcher at the top for **Faculty**, **Student** and **Master Admin**, as shown in Figure 2, to a significant degree. All i mean is probably, the input label and validation adjust automatically based on the chosened tab:

- **Faculty** sign in employing their allocated Faculty ID (such as FAC-101) or college email, in many respects.
- **Students** sign in employing their University USN (such as NNM23IS001).
- **Master Admin** signs in with the master email and password created during the setup, in actual application.

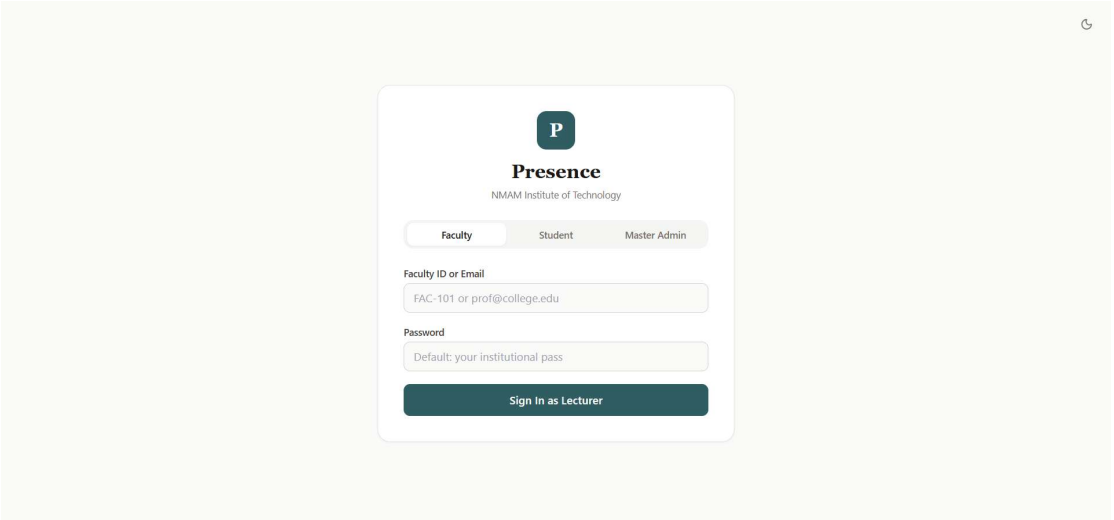


Figure 2, namely *Role-based Login Portal showing the Faculty, Student and Master Admin tabs*

6. Faculty (Lecturer) Module

6.1 Rapid Lecture Marking (Grid View vs. List View)

At the top, the lecturer picks the course, the date, and the **Session Weight** (1 Hr (Theory) or 2 Hrs (Lab)), in real-world scenarios; indeed, once a lecturer logs in, the Mark Attendance screen loads their allotted courses; in turn, plus there's an optional text box to record the topic covered in class, in real-world scenarios.

- **Dozens of students fit on screen at once without scrolling. Roll Number Grid View:** Each student is shown as a compact square card showing the last three digits of their USN (like 001, 002, 042), their first name, and their status badge, in real-world scenarios.
- **Student List View, namely** A classic row-by-row table showing the full USN, full name, and P / A buttons, broadly speaking.

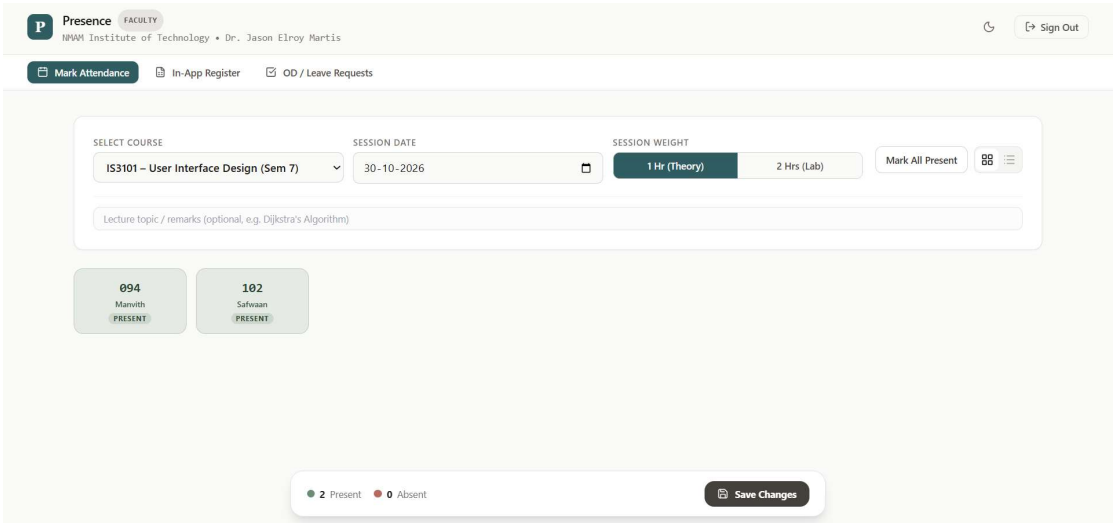


Figure 3, namely *Faculty Mark Attendance screen in Roll Number Grid mode*

6.2 Exception Marking and Explicit Save Bar

In a normal college lecture, most students are, more accurately, present. What's more, only a few are currently absent. Whenever a course is chosen perhaps, the app automatically is marking every student in that batch as **Present** (sage green #6E8B74), in real-world scenarios. The teacher only has to tap the roll numbers of the students who are normally absent, which flips their cards to **Absent** (clay red #B86B61), on the whole.

Attendance is only committed when the teacher clicks Save Changes, preventing accidental taps from changing entries prematurely, in many respects; and thus, a floating bar displays the live count of Present and Absent students along with a **Save Changes** button at the bottom of the screen, in actual application.

6.3 In-App Attendance Register and CSV Export

Teachers had to download a CSV file just to check past attendance in early prototypes, to a significant degree. When you want to fix this, I built the **In-App Register** tab; so, it renders a full spreadsheet matrix inside the browser (Figure 4):

- **Sticky Columns, namely** The student's USN and Student Name stay pinned on the left side while scrolling horizontally, across the weeks of class dates [9].
- **Session Headers:** Each column header shows the date (DD/MM), the class number, and the hour weight (1h or 2h), in many respects.
- **Shortage Filter, namely** Clicking the **Shortage (<75%)** button is hiding eligible students and immediately lists only the students at risk of detention, in many respects.
- **Download CSV -** Clicking **Download CSV** is creating an formal attendance sheet including course metadata, every class date, attended hours, conducted hours, percentage and eligibility status [8], broadly speaking.

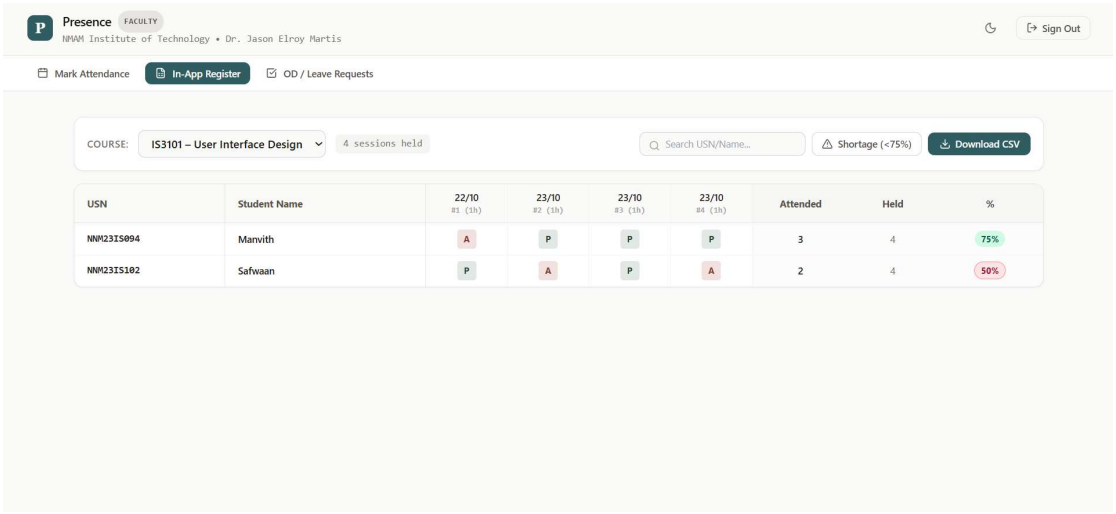


Figure 4, including *In-App Attendance Register with sticky USN columns and the Shortage filter*

7. Student Module

7.1 Student Dashboard

The dashboard is showing their overall attendance percentage, total hours attended, and total hours conducted across all subjects in their semester, when a student logs in with their USN, to a significant degree; indeed, below the summary banner, each subject is showed as a card with a colour-coded progress bar (green for ≥ 75%, red for < 75%) and its Bunk Buffer message, as shown in Figure 5, to a significant degree.

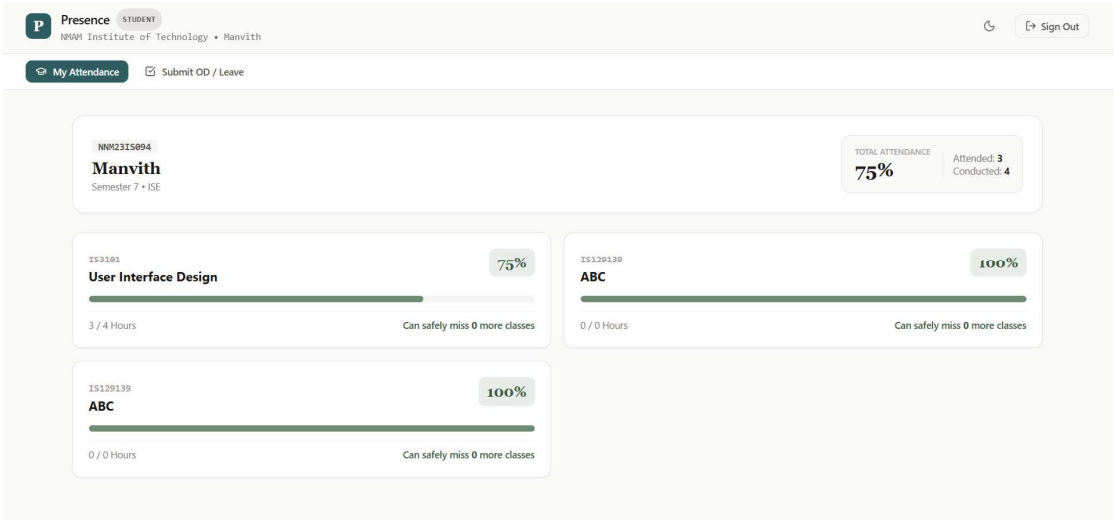


Figure 5, namely *Student Dashboard showing subject cards and Bunk Buffer metrics*

7.2 Submitting On-Duty (OD) and Medical Leave

Below the for**m, students can** (in practice) track whether their previous applications are PENDING, Authorized, or REJECTED, to a significant degree. On the Submit OD / Leave tab, a course can be selected by students, pick the date of absence, choose the category (On-

Duty for fests/hackathons/sports or Medical Leave), and type their justification, in actual application.

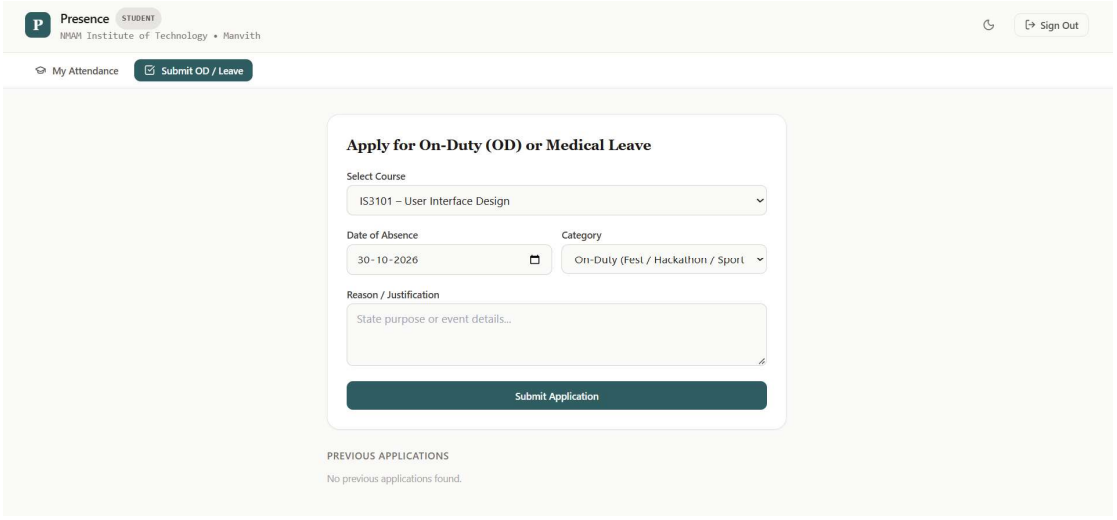


Figure 6 - *Student OD / Medical Leave submission form and application history*

8. Master Admin (Department) Module

8.1 Institute Overview and Threshold Control

The Master Admin dashboard shows four summary cards, including Enrolled Students, Faculty Members, Active Courses, and Classes Conducted; it also is offering a control to adjust the minimum attendance threshold percentage across the institution (Figure 7), in many respects.

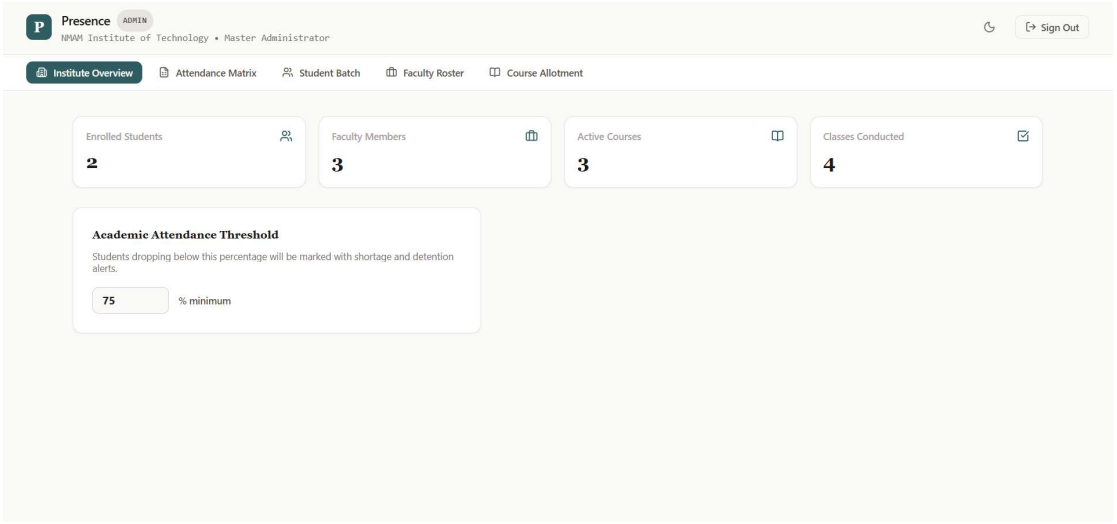


Figure 7, including *Master Admin Institute Overview with the attendance threshold control*

The **Attendance Matrix** tab is giving the admin the same register that faculty see, but for every course in the college, with the same shortage filter and CSV download (Figure 8), in real-world scenarios.

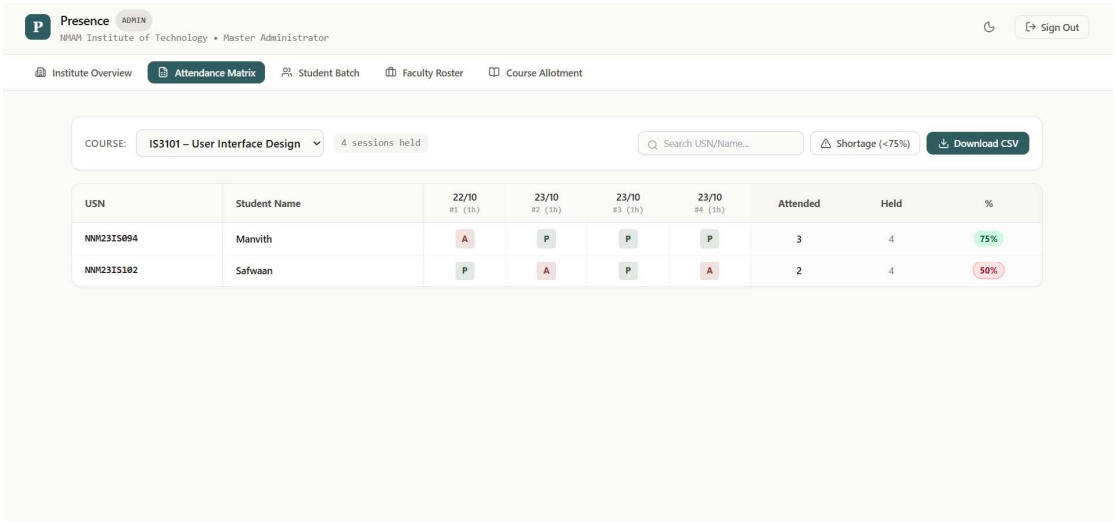


Figure 8 - *Master Admin Attendance Matrix for a picked course*

8.2 Student Batch Manager (Single & Bulk CSV Import)

Typing 60 students one by one takes too long, so the **Student Batch** tab includes a **+ Bulk Batch Import (CSV lines)** toggle, alongside the single-student form (Figure 9), on the whole; the target department and semester are chosen by the admin, pastes lines in the format USN, Student Name (to take one instance, NNM23IS001, Aarav Sharma), and clicks **Read & Enroll Batch** to register the entire class at once, broadly speaking.

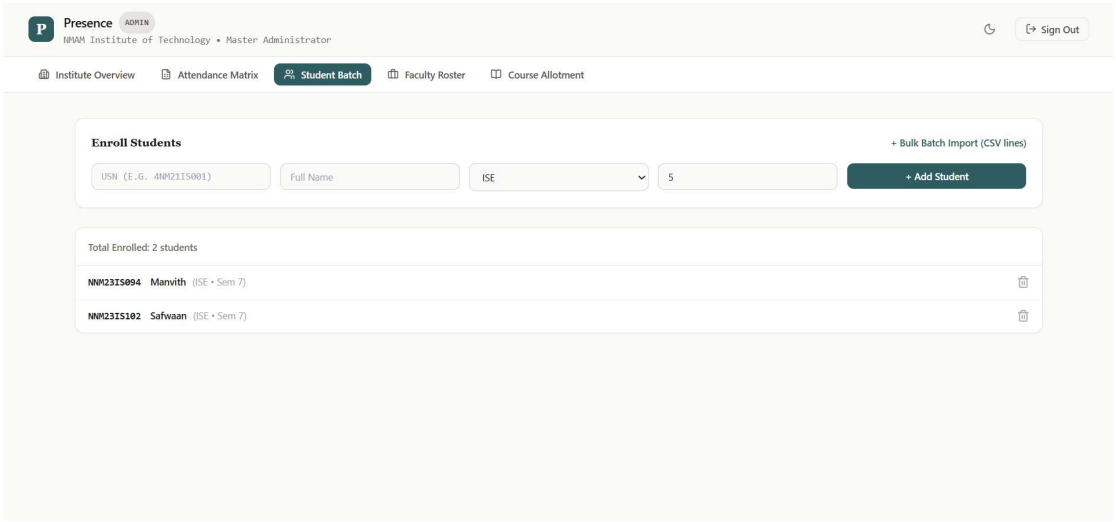


Figure 9, such as *Master Admin Student Batch screen with single enrollment and bulk import option*

8.3 Faculty Roster and Course Allotment

Under **Faculty Roster**, the admin onboards lecturers with their unique Faculty ID (FAC-101), name and email, with a focus on branch (Figure 10), broadly speaking. Under **Course Allotment**, the admin is creating subjects (21CS51, Computer Networks), assigns the semester, and maps the subject to a faculty member (Figure 11), in real-world scenarios.

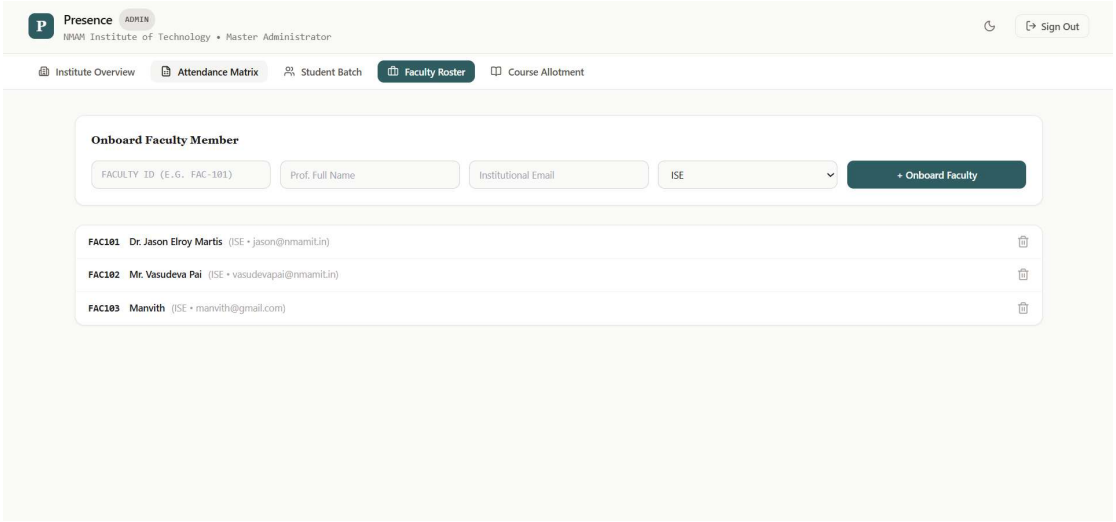


Figure 10, including *Master Admin Faculty Roster for onboarding lecturers*

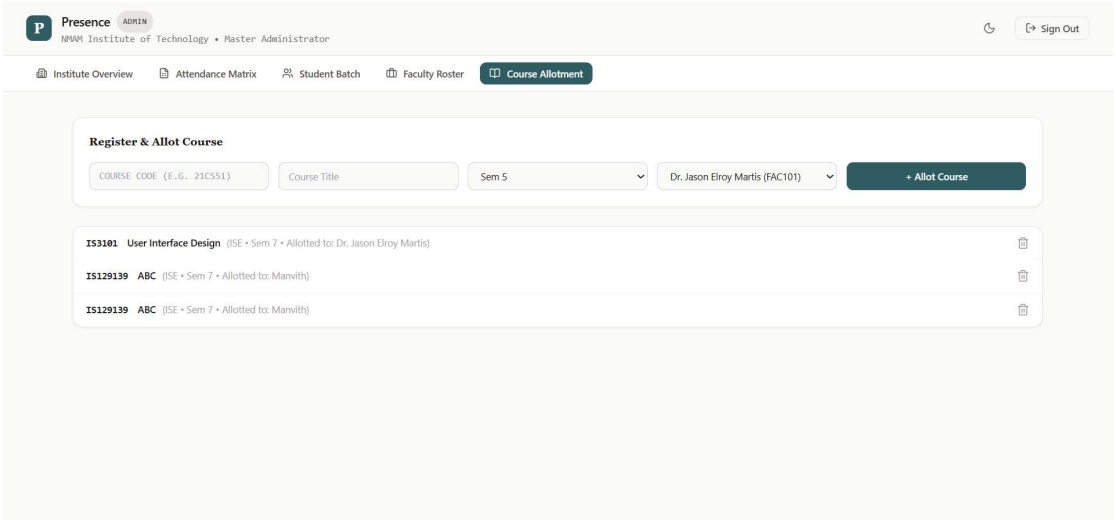


Figure 11, such as *Master Admin Course Allotment screen mapping courses to faculty*

9. Attendance Calculation, Bunk Buffer and OD Workflow

9.1 Weighted Percentage Formula

For any student in a course with N conducted sessions, where session k has weight w_k □ {1, 2}, the hours and percentage are often found as follows:

$$\text{Conducted Hours } (H_c) = \Sigma\, w_k \qquad \text{for k = 1 to N}$$

$$\text{Attended Hours } (H_a) = \Sigma\, w_k \times I(status_k \text{ □ } \{\text{P, OD}\})$$

$$\text{Percentage } (P) = 100\% \qquad \text{if } H_c = 0$$

$$\text{Percentage } (P) = \text{round}(\, H_a \div H_c \times 100\,) \qquad \text{if } H_c > 0$$

Here I(·) is an indicator that equals 1 when the student is generally marked Present (P) or On-Duty (OD) in session k, and 0 otherwise, in actual application.

9.2 Bunk Buffer and Recovery Target Algorithm

1. **Bunk Buffer (when P ≥ T, where T = 75):** finds the maximum number of future 1-hour classes B a student can miss while staying at or above 75%:

$$B = \lfloor (H_a - 0.75 \times H_c) \div 0.75 \rfloor$$

2. **Recovery Target (when P < T):** finds the minimum number of consecutive 1-hour classes R the student must attend to reach 75%:

$$R = \lceil (0.75 \times H_c - H_a) \div (1 - 0.75) \rceil$$

For one, a student with 4 of 5 hours attended has a Bunk Buffer of $\lfloor (4 - 3.75) \div 0.75 \rfloor = 0$ classes, in real-world scenarios, and at the moment, a student with 2 of 5 hours must attend $\lceil (3.75 - 2) \div 0.25 \rceil = 7$ consecutive classes to recover to 75%, to a significant degree.

10. UI Design and Dark Mode Implementation

10.1 Visual Palette

The interface is employing a warm off-white background (#FAFAF8) in light mode and dark charcoal (#121214 and #1C1C1E) in dark mode, paired with a deep teal accent (#2F5D62), broadly speaking; status colors are mainly muted so they're easy on the eyes, like sage green (#6E8B74) for Present and clay red (#B86B61) for Absent, in actual application.

10.2 Class-Based Dark Mode Fix

As a first move, Tailwind CSS v3 depends on the operating system's theme (prefers-colour-scheme), which prevented the in-app sun/moon toggle button from switching themes, on the whole. To fix this, darkMode: 'class' was added to tailwind.config.js [5]; indeed, in localStorage, a useEffect hook in App.tsx syncs the .dark class on document.documentElement (<html>) and saves the choice, on the whole.

11. Testing and Results

11.1 Manual Verification Scenarios

The complete process was tested locally on Vite (http://localhost:5173) from an uninitialized browser state through the admin setup, batch enrollment, lecture marking, student shortage checks, and CSV export, broadly speaking.

Table 6, such as Manual test scenarios and findings

Scenario	Expected Result	Observed
Let's get going launch with empty localStorage	Opens Institutional Setup Wizard automatically	Pass
Admin pastes 10 CSV lines in Bulk Student Import	Scans all 10 USNs and enrolls them into the chosen semester	Pass

Scenario	Expected Result	Observed
Lecturer picks a 2-Hr Lab slot and marks 1 student Absent	Conducted hours increase by 2; absent student is losing 2 hours	Pass
Student has 4/5 hours attended (80% with 75% threshold)	Displays Bunk Buffer: "Can safely miss 0 more classes"	Pass
Student has 2/5 hours attended (40% with 75% threshold)	Displays Shortage Alert: "Attend next 7 classes to reach 75%"	Pass
Lecturer clicks "Shortage (<75%)" then "Download CSV"	Exports CSV comprising only students below 75%	Pass
Clicking Sun/Moon button in top header	Immediately toggles .dark class on <html> and keeps going on reload	Pass

12. Local Setup and Rollout

The project runs locally in Visual Studio Code with Node.js and Vite in under the three minutes:

1. **Scaffold project:** npm are creating vite@latest presence-app -- --template react-ts
2. **Install packages, such as** npm install lucide-react and npm install -D tailwindcss@3 postcss autoprefixer
3. **Initialize Tailwind, such as** npx tailwindcss init -p and set darkMode: 'class' inside tailwind.config.js.
4. **Run dev server:** the local server on http://localhost:5173 is being started by npm run dev, on the whole.
5. **Production build -** npm run build compiles the TypeScript and CSS into the static assets in dist/, which can (in practical terms) be rolled out directly to Vercel, Netlify, or Render Static Sites, in actual application.

13. Conclusion and Future Scope

13.1 Conclusion

Building **Presence** showed how much faster classroom software becomes when unnecessary attributes are removed, to a significant degree. Dropping rotating QR codes and "Late" statuses in favor of a default-Present Roll Number Grid is allowing a lecturer to record attendance for an entire class in seconds, on the whole; adding the in-app register table alongside the CSV export lets faculty and the HOD inspect attendance directly in the browser while keeping formal spreadsheet exports one click away.

13.2 Future Scope

- Connecting the frontend to a Node.js/Express and PostgreSQL/MongoDB backend so data syncs, across the many devices, broadly speaking.
- Combining timetable slots so the lecturer's current period opens automatically based on the time of day, broadly speaking.
- Adding automated email or SMS alerts to parents when a student's attendance drops below 65%, in real-world scenarios.
- Creating printable PDF shortage notices directly from the register view.

14. Disclosure of AI Assistance

During this project, in line with the course policy, Gemini (Google) was employed as a programming assistant; it helped create the TypeScript interfaces and component arrangement in src/App.tsx, assisted in writing the CSV Blob export logic, and helped troubleshoot the Tailwind CSS v3 CLI installation and darkMode: 'class' setup in PowerShell, to a significant degree.

The code and calculations well enough are understood to explain and change them. My part was to specify the college-specific process, remove impractical traits (such as the QR code system and “Late” status), stipulate the Present/Absent exception-marking design with an unambiguous Save button, need the in-app register table alongside CSV downloads, test the application in VS Code, and prepare this report, on the whole.

15. References

[1] NMAM Institute of Technology. *Academic Regulations and Curriculum*. [Online]. Available: https://nitte.edu.in/nmamit/

[2] Meta Open Source. *React Documentation*. [Online]. Available: https://react.dev/

[3] Vite. *Getting Started with Vite*. [Online]. Available: https://vite.dev/guide/

[4] Microsoft. *TypeScript Handbook*. [Online]. Available: https://www.typescriptlang.org/docs/handbook/intro.html

[5] Tailwind Labs. *Tailwind CSS v3 Documentation - Dark Mode*. [Online]. Available: https://v3.tailwindcss.com/docs/dark-mode

[6] Lucide Contributors. *Lucide React Icon Library*. [Online]. Available: https://lucide.dev/guide/packages/lucide-react

[7] MDN Web Docs. *Window: localStorage Property*. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[8] MDN Web Docs. *Blob API, an application programming interface, and URL.createObjectURL()*. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Blob

[9] MDN Web Docs. *CSS Positioned Layout, namely position: sticky*. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/position

16. Academic Integrity Statement

"I declare that I have personally tested and understood the submitted project. I have disclosed the AI tools and external resources used in developing it. I can explain the architecture, the role-based views, the attendance and bunk-buffer calculations, and the modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own."

Name: Manvith D Shettigar

USN: NNM23IS094

Signature: 

Date: 30-10-2026